

Diagramas UML

CodeByMike

Michael David Rodríguez Beltrán

Análisis y Desarrollo de Software (ADSO)

Ficha 3114731

Servicio Nacional de Aprendizaje (SENA)

Centro de Servicios Financieros, Regional Distrito Capital

Instructora: Diana Delgado

Agosto de 2026

Tabla de contenido

Introducción

Objetivos

Marco conceptual

Descripción del sistema modelado

Diagramas de comportamiento

Diagrama de casos de uso

Documentación de los casos de uso

Diagrama de secuencia

Diagrama de comunicación

Diagrama de actividades

Diagramas de estructura

Diagrama de clases

Diagrama de objetos

Diagrama de componentes

Diagrama de despliegue

Diagrama de paquetes

Conclusiones

Referencias

Introducción

El lenguaje unificado de modelado (UML, por sus siglas en inglés) surgió en 1994 del trabajo conjunto de Rumbaugh, Booch y Jacobson, y fue adoptado como estándar por el Object Management Group en su versión 1.1 de 1997 (Servicio Nacional de Aprendizaje [SENA], 2018). Su propósito es representar de manera gráfica y estandarizada los modelos de un sistema, de modo que un diseño pueda compartirse entre distintos diseñadores y entre el equipo de desarrollo y el cliente sin depender de un lenguaje de programación concreto (Rumbaugh et al., 2007).

Este documento reúne los diagramas UML elaborados para el sistema CodeByMike, una plataforma que integra un portafolio público, un panel de control administrativo, un portal de clientes y un laboratorio de ingeniería. Cada diagrama se acompaña de la definición del tipo al que pertenece, la notación empleada y una interpretación de lo que el modelo revela sobre el sistema, siguiendo la secuencia pedagógica propuesta en la guía Introducción al lenguaje de modelado UML (SENA, 2018).

Los diagramas no se dibujaron en una herramienta externa: se generan desde el propio código fuente del sistema, ya sea mediante Mermaid o mediante motores de trazado escritos para el proyecto, y se exportaron a este documento directamente desde la aplicación en ejecución. Esa decisión garantiza que el modelo aquí presentado corresponda al sistema tal como está construido y no a una versión que quedó desactualizada al primer cambio de código.

Objetivos

Objetivo general

Documentar el análisis y el diseño del sistema CodeByMike mediante los diagramas del lenguaje unificado de modelado, de manera que la estructura y el comportamiento de la solución queden representados en una notación estándar y verificable.

Objetivos específicos

- Identificar los actores y las funcionalidades del sistema a través de diagramas de casos de uso.
- Describir el comportamiento dinámico de las funcionalidades críticas con diagramas de secuencia, comunicación y actividades.
- Representar la estructura estática de la solución con diagramas de clases, objetos, componentes, despliegue y paquetes.
- Relacionar cada diagrama con el elemento del código fuente que lo implementa, para conservar la trazabilidad entre el modelo y el sistema construido.

Marco conceptual

UML es un lenguaje centrado en la representación gráfica de un sistema, que indica cómo crear y leer los modelos representando flujos de trabajo mediante elementos gráficos (SENA, 2018). Sus cuatro propósitos son visualizar el sistema de forma que otros lo comprendan, especificar sus características antes del desarrollo, construir los sistemas diseñados a partir de los modelos y documentar la solución para facilitar su mantenimiento posterior.

Un modelo UML se compone de tres bloques de construcción. Los elementos son abstracciones de cosas reales o ficticias, con identidad propia y distinguibles entre sí; las relaciones vinculan esos elementos y dotan de funcionalidad al sistema; los diagramas reflejan gráficamente el comportamiento y las relaciones entre elementos (SENA, 2018). La especificación vigente del estándar, UML 2.5.1, clasifica los diagramas en dos grandes familias: los de estructura, que describen la organización estática del sistema, y los de comportamiento, que describen su dinámica (Object Management Group [OMG], 2017).

Esa clasificación organiza el presente documento. Primero se presentan los diagramas de comportamiento, porque el análisis parte de lo que el sistema debe hacer y de quién lo solicita; después los de estructura, que responden a cómo se organiza internamente para hacerlo. Larman (2007) sostiene precisamente que el orden natural del análisis orientado a objetos va del comportamiento observable hacia la asignación de responsabilidades entre clases.

Descripción del sistema modelado

CodeByMike es un sistema de información desplegado en producción bajo el dominio codebymike.tech. Reúne cuatro subsistemas que comparten una misma base de datos y una misma capa de seguridad: un portafolio público con contenido técnico, un panel de control administrativo que gestiona clientes, proyectos, costos y cobros, un portal privado donde cada cliente consulta el estado de sus proyectos y sus facturas, y un laboratorio de ingeniería que expone las pruebas, los indicadores de nivel de servicio y la observabilidad de seguridad del propio sistema.

El sistema se construyó con Astro sobre renderizado del lado del servidor, una base de datos libSQL gestionada con Drizzle, y se despliega sobre una plataforma de cómputo sin servidor. Tres mecanismos de autenticación conviven sin compartir estado: el acceso administrativo mediante OAuth con lista de permitidos, el portal de clientes con credenciales propias, y un pase temporal para la demostración pública de solo lectura. Esta separación, que resulta invisible en la interfaz, es una de las decisiones de diseño que los diagramas hacen explícita.

Los diagramas que siguen no describen un sistema hipotético: cada actor, cada clase y cada nodo de despliegue corresponde a un elemento existente en el código fuente o en la infraestructura de producción. Bajo cada figura se indica, cuando aplica, el módulo que la implementa.

Diagramas de comportamiento

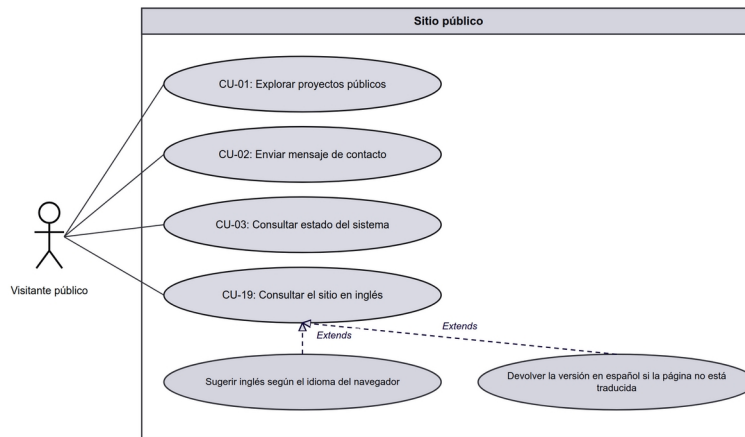
Diagrama de casos de uso

El diagrama de casos de uso muestra las relaciones entre los actores y el sistema, y modela la funcionalidad según la percepción del usuario externo (SENA, 2018). Es responsable de documentar los macrorrequisitos: puede leerse como la lista de funcionalidades que el sistema debe proporcionar. Fowler y Scott (1997) advierten que su valor no está en el dibujo sino en la disciplina de delimitar qué queda dentro y qué queda fuera de la frontera del sistema.

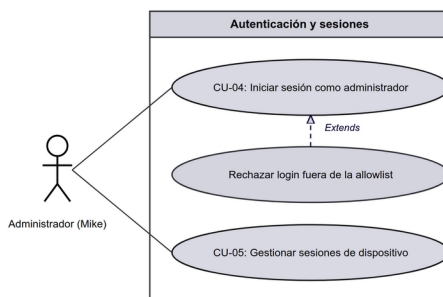
La notación emplea tres componentes. El sistema se representa mediante un rectángulo que delimita gráficamente lo que está dentro (los casos de uso) y lo que está fuera (los actores). El actor se representa mediante una figura humana esquemática e indica el tipo de usuario que podrá ejecutar alguna función. El caso de uso se representa mediante un óvalo y se nombra con un verbo en infinitivo que expresa la función a realizar (SENA, 2018).

Entre esos elementos se dan tres tipos de relaciones. La asociación, trazada del actor al caso de uso, indica que el actor lleva a cabo esa funcionalidad. La relación de inclusión, marcada con el estereotipo <<include>>, se da cuando un caso de uso incorpora obligatoriamente el comportamiento de otro. La relación de extensión, marcada con <<extend>>, señala que un caso de uso amplía o especializa a otro en circunstancias determinadas.

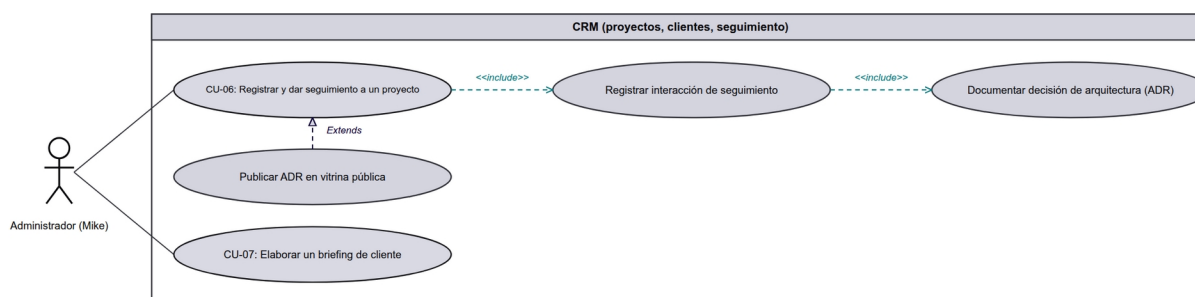
Las figuras siguientes presentan el diagrama de casos de uso del sistema, una por subsistema: cada figura muestra la frontera de un módulo, los casos de uso que contiene y el actor que los ejecuta. Se presentan por separado, y no como un único diagrama, porque el conjunto completo no cabe en una hoja con un tamaño de letra legible; la lectura conjunta se obtiene recorriéndolas en orden. Los actores identificados son el administrador, el cliente, el visitante público y los sistemas externos que actúan sin intervención humana, como el planificador de tareas programadas y la pasarela de pagos. Su inclusión como actores obedece a que inician casos de uso por su cuenta: la técnica no exige que un actor sea una persona, sino una entidad externa a la frontera del sistema.

Figura 1*Sitio público - Visitante público*

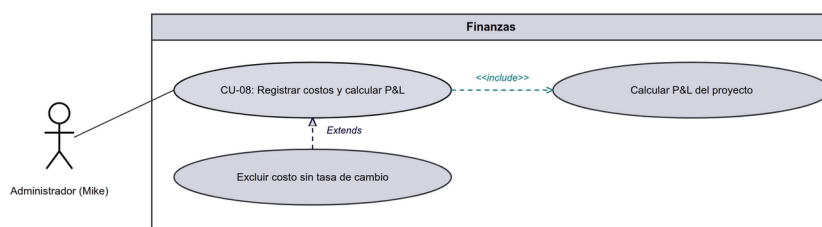
Nota. CU-19 declara dos extensiones y ninguna inclusión. Sugerir el inglés según el idioma del navegador solo ocurre si el visitante llega con otro idioma configurado, y devolver la versión en español solo si esa página aún no está traducida; en ambos casos el caso base se completa sin ellas. Un comportamiento que siempre se ejecuta se modela como <<include>>, uno que depende de una condición como <<extend>>.

Figura 2*Autenticación y sesiones - Administrador (Mike)*

Nota. Rechazar el login fuera de la allowlist extiende a CU-04 porque solo se activa cuando la validación de la lista de permitidos falla. El flujo principal de autenticación termina sin pasar nunca por esa extensión, que es la razón por la que no se modela como inclusión.

Figura 3*CRM (proyectos, clientes, seguimiento) - Administrador (Mike)*

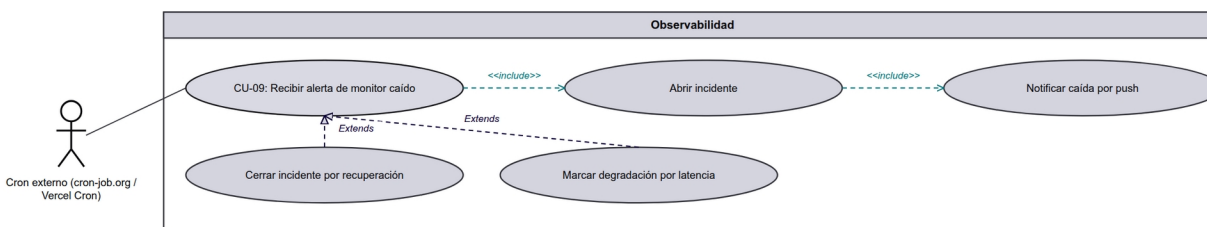
Nota. CU-06 incluye dos pasos encadenados, registrar la interacción de seguimiento y documentar la decisión de arquitectura: son <<include>> porque el seguimiento de un proyecto no está completo sin ellos, y van encadenados porque el ADR se documenta sobre una interacción ya registrada. Publicar el ADR en la vitrina pública, en cambio, extiende al caso base: la decisión puede quedarse privada y el caso de uso igual se cumple.

Figura 4*Finanzas - Administrador (Mike)*

Nota. Calcular el P&L del proyecto es un <<include>> de CU-08: registrar un costo sin recalculer el resultado dejaría el proyecto con cifras inconsistentes, de modo que el paso no es opcional. Excluir un costo sin tasa de cambio es un <<extend>> porque solo se dispara ante el caso particular de un costo en otra moneda sin tasa registrada.

Figura 5

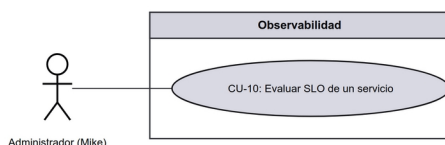
Observabilidad - Cron externo (cron-job.org / Vercel Cron)



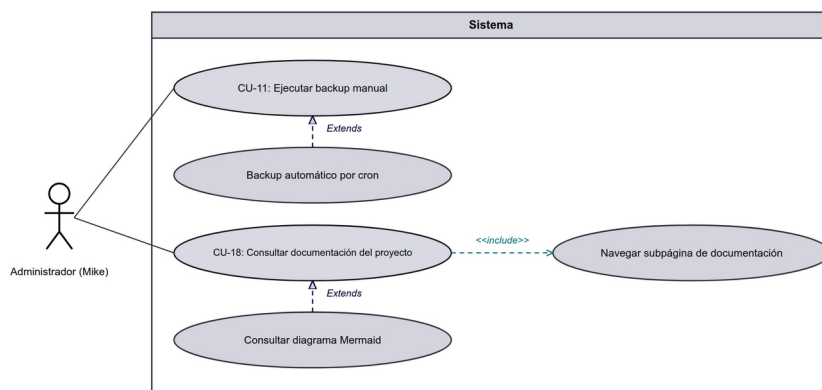
Nota. Abrir el incidente y notificar la caída por push son inclusiones encadenadas: cuando el chequeo detecta la caída, ambos pasos se ejecutan siempre y en ese orden, porque la notificación necesita el incidente ya creado. Cerrar el incidente por recuperación y marcar la degradación por latencia son extensiones: dependen de condiciones distintas (que el servicio vuelva, o que responda pero lento) y ninguna forma parte del flujo básico de la alerta.

Figura 6

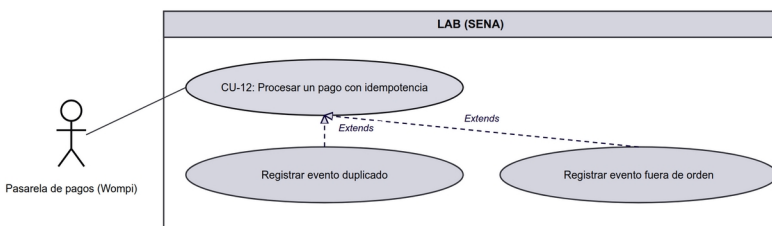
Observabilidad - Administrador (Mike)



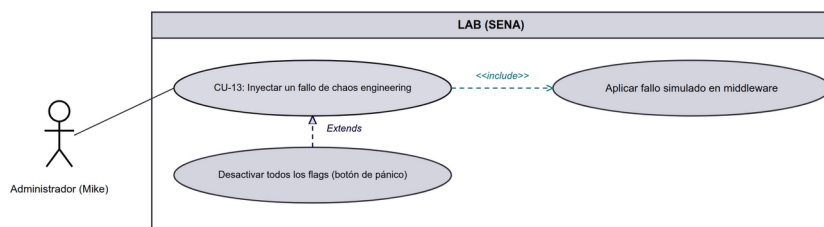
Nota. CU-10 no declara relaciones. Es un caso de uso atómico: consultar el presupuesto de error de un monitor no exige ningún paso compartido con otros casos ni admite variantes condicionales, y descomponerlo solo añadiría ruido al diagrama.

Figura 7*Sistema - Administrador (Mike)*

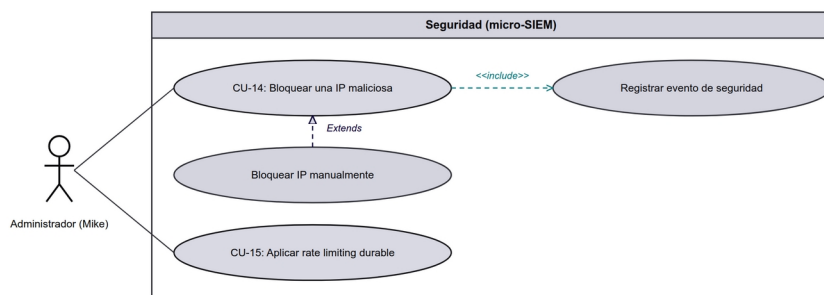
Nota. El backup automático por cron extiende a CU-11 porque es la misma operación disparada por otro camino, el planificador externo en lugar del administrador, y solo ocurre en esa circunstancia. En CU-18, navegar una subpágina de documentación es una inclusión (no hay consulta sin navegación) y consultar un diagrama Mermaid es una extensión, ya que solo algunas páginas de la documentación llevan diagrama.

Figura 8*LAB (SENA) - Pasarela de pagos (Wompi)*

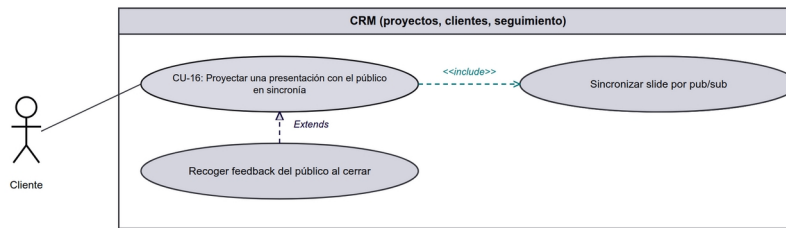
Nota. CU-12 tiene dos extensiones y ninguna inclusión, y eso describe con precisión el diseño de idempotencia: el flujo normal aplica el evento del webhook y termina. Registrar un evento duplicado ocurre solo si la pasarela reenvía un evento ya procesado, y registrar un evento fuera de orden solo si llega uno anterior al estado actual. Ambos son desvíos condicionales, no pasos del camino feliz.

Figura 9*LAB (SENA) - Administrador (Mike)*

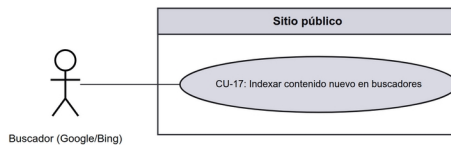
Nota. Aplicar el fallo simulado en el middleware es un <<include>> de CU-13: sin ese paso el flag quedaría registrado pero no habría fallo que observar, luego el paso es parte constitutiva del caso. Desactivar todos los flags con el botón de pánico extiende al caso base porque es una salida de emergencia que solo se usa si el experimento se descontrola.

Figura 10*Seguridad (micro-SIEM) - Administrador (Mike)*

Nota. Registrar el evento de seguridad es una inclusión de CU-14 sin excepciones: toda decisión del sensor queda en la bitácora del micro-SIEM, tanto si termina en bloqueo como si no, y una bitácora con huecos no serviría para auditar. Bloquear la IP manualmente extiende al caso porque es la vía alternativa a la que el auto-bloqueo recorre por su cuenta.

Figura 11*CRM (proyectos, clientes, seguimiento) - Cliente*

Nota. CU-16 incluye generar un PIN libre de colisiones y sincronizar la diapositiva por pub/sub: sin cualquiera de los dos no hay sesión proyectada, así que son obligatorios. Recoger el feedback del público al cerrar es una extensión porque ocurre al final y solo si la sesión se cierra formalmente.

Figura 12*Sitio público - Buscador (Google/Bing)*

Nota. CU-17 no declara relaciones. El actor es un sistema externo (el rastreador del buscador) y el caso de uso se agota en un intercambio: se notifica el contenido nuevo y se actualizan los recursos de indexación, sin pasos compartidos ni variantes condicionales.

Documentación de los casos de uso

La técnica de casos de uso exige, además del diagrama, la descripción de cada caso: el flujo de eventos que se da entre el sistema y el actor (SENA, 2018). Las tablas siguientes recogen ese formato extendido (precondiciones, secuencia normal, flujos alternos, excepciones y postcondiciones) para los casos de uso más críticos del sistema, que son los que involucran dinero, seguridad o estado que debe permanecer consistente ante fallos.

Tabla 1

Descripción del caso de uso CU-04: Iniciar sesión como administrador (actor: Administrador (Mike))

Descripción	El administrador se autentica con GitHub y accede al panel si su login está en la allowlist.
Precondición	El administrador tiene una cuenta de GitHub válida. Su login está registrado en la allowlist de src/lib/auth.ts.
Secuencia normal	1. El administrador visita /admin sin sesión activa. 2. El middleware detecta ausencia de sesión y redirige a /api/auth/signin. 3. El administrador autoriza la app OAuth de GitHub. 4. Auth.js valida el login contra la allowlist. 5. Se emite un JWT de sesión y se registra el dispositivo en admin_sessions. 6. El administrador es redirigido al panel /admin.
Flujos alternos	1. GitHub autentica correctamente pero el login no está en la allowlist. 2. Auth.js rechaza la sesión y muestra error de acceso denegado.
Postcondición	El administrador tiene una sesión JWT activa y un registro en admin_sessions con IP y user-agent.
Excepciones	GitHub OAuth no disponible: el login falla con mensaje de error genérico, sin exponer detalles internos.

Tabla 2

Descripción del caso de uso CU-09: Recibir alerta de monitor caído (actor: Cron externo (cron-job.org / Vercel Cron))

Descripción	El cron externo dispara el chequeo, detecta una caída, abre un incidente y notifica por push.
Precondición	Existe un monitor activo y no pausado en la tabla monitors. El cron externo tiene configurado el CRON_SECRET válido.
Secuencia normal	1. El cron externo llama a /api/cron/uptime-check con el secreto. 2. El sistema itera los monitores activos y hace la petición HTTP configurada (método, texto esperado, umbral de latencia). 3. La respuesta falla (status inesperado, timeout o texto ausente). 4. Se inserta un monitor_check con ok=false. 5. Si es el primer fallo consecutivo, se abre un monitor_incidents con startedAt. 6. Se actualiza monitors.lastStatus a "down" y se dispara una notificación push (ntfy).
Flujos alternos	1. Un chequeo posterior tiene éxito. 2. Se cierra el incidente abierto con resolvedAt y durationSec. 3. Se notifica la recuperación. 4. La respuesta es exitosa pero supera latencyThresholdMs. 5. lastStatus pasa a "degraded" sin abrir incidente.
Postcondición	El estado materializado del monitor refleja el último chequeo; el historial permite reconstruir el SLO.
Excepciones	El endpoint del monitor no responde en absoluto (timeout de red): se registra como fallo con error de timeout.

Tabla 3

Descripción del caso de uso CU-12: Procesar un pago con idempotencia (actor: Pasarela de pagos (Wompi))

Descripción	La pasarela envía un webhook de pago; el sistema aplica el evento
--------------------	---

	respetando idempotencia y orden.
Precondición	Existe un payment en estado created o pending con un idempotencyKey único.
Secuencia normal	1. La pasarela (Wompi) envía un webhook con el resultado de la transacción. 2. El sistema busca el payment por reference/gatewayTxId. 3. Se registra el evento crudo en payment_events (incluyendo si es duplicado o fuera de orden). 4. Si el evento es válido y en orden, se aplica la transición de estado (created→pending→approved/declined). 5. Se responde 200 a la pasarela para confirmar recepción.
Flujos alternos	1. El gatewayTxId ya fue procesado. 2. Se marca duplicate=true en payment_events. 3. No se modifica el estado del payment. 4. Llega un evento "pending" después de uno "approved". 5. Se marca outOfOrder=true. 6. El estado terminal previo se conserva (nunca retrocede).
Postcondición	El estado del payment refleja fielmente la transacción real, con bitácora completa auditable para sustentación.
Excepciones	El monto del evento no coincide con el del payment: se marca amountMismatch=true y se genera una alerta; el evento nunca se aplica.

Tabla 4

Descripción del caso de uso CU-14: Bloquear una IP maliciosa (actor: Administrador (Mike))

Descripción	El sensor clasifica un request hostil; el administrador (o el auto-block) añade la IP a la blocklist con TTL.
Precondición	El sensor de seguridad (sensor.ts) está observando requests entrantes.
Secuencia	1. Llega un request al middleware.

normal	2. observeRequest clasifica el request contra las firmas conocidas (classify.ts).
	3. Se detecta una firma de severidad alta/crítica (p. ej. intento de path traversal).
	4. Se registra un security_events con category, severity y ruleId.
	5. El cron de auto-block evalúa la reincidencia de esa IP y decide bloquearla con TTL escalonado (1h → 24h → 7d).
	6. La IP queda en blocked_ips con expiresAt obligatorio.
Flujos alternos	1. El administrador revisa un evento en el panel y decide bloquear la IP manualmente.
	2. Se inserta en blocked_ips con source>manual.
Postcondición	Requests posteriores de esa IP reciben 403 seco hasta que expire el bloqueo.
Excepciones	La lectura de blocklist falla (timeout de DB): el middleware falla abierto y deja pasar el request (nunca bloquea por error interno).

Tabla 5

Descripción del caso de uso CU-18: Consultar documentación del proyecto (actor: Administrador (Mike))

Descripción	El administrador navega /docs para revisar requerimientos, casos de uso, diagramas y el kanban del propio portfolio.
Precondición	El administrador tiene sesión activa en /admin.
Secuencia normal	1. El administrador hace clic en "Documentación" en la sidebar.
	2. Se muestra el hub /docs con visión general, alcance y mapa de subpáginas.
	3. El administrador navega a una subpágina (RF, RNF, CU, diagramas o kanban) usando DocsNav.
	4. La página renderiza el contenido desde src/data/documentacion.ts o src/data/iteraciones-portfolio.ts.

Flujos alternos	1. El administrador entra a una página de diagrama de secuencia, clases u objetos. 2. El navegador renderiza el diagrama Mermaid desde el texto embebido en la página. 3. El administrador entra a una página de diagrama BPMN, de despliegue, de comunicación, de actividades o de componentes. 4. El servidor genera el SVG desde el modelo tipado de src/data/ con el motor de layout correspondiente de src/lib/; el navegador no ejecuta JavaScript para dibujarlo.
Postcondición	El administrador cuenta con la documentación de ingeniería completa del proyecto sin salir del panel.

Tabla 6

Descripción del caso de uso CU-06: Registrar y dar seguimiento a un proyecto (actor: Administrador (Mike))

Descripción	El administrador crea un proyecto, registra interacciones de seguimiento y documenta decisiones de arquitectura.
Precondición	El administrador tiene sesión activa. Opcionalmente existe un cliente en la tabla clients al que asociar el proyecto.
Secuencia normal	1. El administrador crea el proyecto desde /admin/projects con POST /api/admin/projects (requiere slug y title), quedando en estado "activo" y no visible al público. 2. Registra una interacción de seguimiento (llamada, reunión, tarea) con POST /api/admin/interactions, insertando en la tabla interactions con tipo, título, cuerpo y próxima acción. 3. Marca la interacción como resuelta con PUT /api/admin/interactions (done, doneAt). 4. Documenta una decisión de arquitectura con POST /api/admin/projects/[id]/adrs, insertando en project_adrs (contexto, decisión, justificación, estado).

	5. Opcionalmente marca el ADR como isPublic para exponerlo en la vitrina pública del proyecto.
Flujos alternos	1. El administrador cambia el estado con PUT <code>/api/admin/projects/[id]</code> a pausado, completado o archivado. 2. El administrador corrige o elimina una decisión previa vía PUT/DELETE sobre <code>project_adrs</code>.
Postcondición	El proyecto queda con un historial trazable de interacciones y decisiones arquitectónicas en <code>interactions</code> y <code>project_adrs</code> .
Excepciones	El POST de creación llega sin slug o title: la API responde 400 sin tocar la base de datos.

Tabla 7

Descripción del caso de uso CU-08: Registrar costos y calcular P&L (actor: Administrador (Mike))

Descripción	El administrador registra el costo de un servicio, quién lo paga y cuánto se factura al cliente.
Precondición	El proyecto existe. ENCRYPTION_KEY está configurada si el costo incluye credenciales cifradas. Existen tasas de cambio en <code>app_settings</code> para costos que no están en USD.
Secuencia normal	1. El administrador registra un servicio o costo (ciclo de facturación, moneda, quién paga y a quién se factura) insertando en <code>project_services</code>. 2. Registra el ingreso cobrado o pendiente en la tabla <code>finances</code>. 3. La vista del proyecto invoca <code>projectPnL()</code> (<code>src/lib/pnl.ts</code>) con los servicios, las finanzas y las tasas de cambio. 4. <code>projectPnL</code> calcula el costo mensual equivalente en USD por servicio y lo proyecta desde la fecha de inicio del proyecto. 5. Se obtiene el margen estimado restando el costo acumulado a los ingresos cobrados. 6. El panel muestra ingresos, costo mensual/anual, costo

	acumulado y margen, coloreado según sea positivo o negativo.
Flujos alternos	1. El administrador ajusta o borra un costo; el P&L se recalcula en el siguiente render, sin job asíncrono. 2. Un costo en moneda sin tasa configurada se excluye del total y se muestra como advertencia con link a /admin/settings.
Postcondición	El P&L del proyecto refleja el nuevo costo o ingreso desde el siguiente GET del detalle, sin desfase.
Excepciones	Falta ENCRYPTION_KEY al guardar credenciales de un servicio: la API responde 500 pidiendo configurar la clave de cifrado.

Tabla 8

Descripción del caso de uso CU-11: Ejecutar backup manual (actor: Administrador (Mike))

Descripción	El administrador dispara un backup de la base de datos hacia Blob storage desde el panel.
Precondición	El administrador tiene sesión activa (o, en el modo automático, el cron externo dispone del CRON_SECRET). Vercel Blob está habilitado en el proyecto.
Secuencia normal	1. El administrador dispara el backup manual desde /admin/backup. 2. runBackup() consulta en paralelo las tablas de negocio (clients, projects, messages, finances, projectServices, projectAdrs, briefings, entre otras). 3. Arma un dump JSON con metadatos de versión y fecha, más el contenido de cada tabla. 4. Sube el dump a Vercel Blob como backups/portfolio-{fecha}-{timestamp}.json con acceso privado. 5. Devuelve al panel la URL, el tamaño y el pathname del backup generado. 6. El panel lista los últimos 30 backups ordenados por fecha para

	verificación visual.
Flujos alternos	1. Vercel Cron llama al mismo endpoint con el CRON_SECRET en lugar de sesión de administrador, ejecutando runBackup() sin intervención manual. 2. El administrador solo lista los backups existentes, sin generar uno nuevo.
Postcondición	Queda un archivo JSON inmutable en Vercel Blob con una fotografía completa de las tablas de negocio en ese momento.
Excepciones	Falla la conexión a la base de datos durante el respaldo: el endpoint responde 500 y el fallo queda registrado en logs, sin generar un blob parcial.

Tabla 9

Descripción del caso de uso CU-13: Inyectar un fallo de chaos engineering (actor: Administrador (Mike))

Descripción	El administrador activa un flag de fallo temporal en una ruta y observa cómo el monitoreo lo detecta.
Precondición	El administrador tiene sesión activa. La ruta objetivo no pertenece a /admin, /api/admin ni /api/auth (protegidas contra auto-sabotaje).
Secuencia normal	1. El administrador crea un flag de chaos desde /admin/lab/chaos, indicando tipo (latencia, error 500 o caída de servicio), ruta objetivo y TTL. 2. El sistema valida el tipo y la ruta, aplica topes de seguridad (latencia máxima y TTL máximo) y calcula la expiración. 3. El flag se inserta activo en la tabla chaos_flags y se invalida la caché para que aplique de inmediato. 4. En cada request, el middleware evalúa los flags activos (con caché corta) y busca una coincidencia con la ruta solicitada. 5. Si coincide, aplica el fallo simulado: introduce latencia, o responde error 500/503 con un header que identifica que es

	chaos.
	6. El chequeo de uptime del cron detecta la caída simulada en su siguiente sondeo, igual que detectaría una caída real.
Flujos alternos	1. Al vencer el TTL, el flag deja de aplicarse automáticamente, sin que el administrador tenga que desactivarlo. 2. El administrador desactiva todos los flags activos de una sola vez desde el panel.
Postcondición	Las rutas coincidentes sufren el fallo simulado hasta que el flag expira o se apaga manualmente, permitiendo validar que el monitoreo lo detecta.
Excepciones	Si la lectura de flags falla por un problema de base de datos, el middleware falla abierto: el request pasa limpio y nunca se cae el sitio real por un error del propio motor de caos.

Tabla 10

Descripción del caso de uso CU-16: Proyectar una presentación con el público en sincronía (actor: Cliente)

Descripción	El administrador proyecta un deck y lo controla desde su celular; el público lo sigue en sus propios dispositivos entrando por un QR o un PIN de cuatro caracteres.
Precondición	Existe un deck en la biblioteca: un archivo HTML autónomo con un <deck-stage> del que se extrajeron sus slides al subirlo.
Secuencia normal	1. El administrador pulsa Presentar y confirma en una pantalla que muestra el deck, su número de slides y la caducidad de la sesión. 2. El sistema crea la sesión en Redis en estado lobby, con slide 0 y un PIN de cuatro caracteres (dos letras y dos dígitos) comprobado contra las rutas reservadas del sitio y contra los PIN ya en uso. 3. La pantalla de reparto ofrece las dos vistas: la pantalla principal

para el proyector y el control remoto, este último también como QR para escanearlo con el celular.

4. La pantalla principal muestra a pantalla completa el QR hacia `codebymike.tech/{pin}` y el PIN escrito en grande; es la única vista que los muestra.
5. El público escanea o teclea la dirección y ve la pantalla de espera con el título del deck.
6. El administrador inicia desde el control remoto, que exige sesión de administrador y valida además el secreto de la sesión.
7. Cada comando (anterior, siguiente, salto directo) se valida en el servidor contra el rango de slides, se persiste en Redis y se publica al bus.
8. Cada dispositivo del salón, suscrito directamente al bus, recibe el cambio y salta al slide correspondiente en menos de 300 ms.

Flujos alternos

1. Al conectar, el cliente pide el snapshot de la sesión y entra directamente al slide en curso, sin ver los anteriores.
2. El administrador abre el selector de slides del control remoto y salta a uno concreto por su número y rótulo, en lugar de avanzar de a uno.
3. Al pasar del último slide o pulsar Finalizar, las tres vistas muestran la misma pantalla de cierre con un QR hacia `/feedback`.

Postcondición Al terminar, la sesión pasa a estado ended y libera su PIN, que vuelve a quedar disponible para otra sesión. El estado efímero caduca solo por TTL sin dejar rastro en la base de datos.

Excepciones

1. Si se pierde la red del celular, al reconectar el control retoma el slide real: el servidor es la fuente de verdad y el cliente nunca impone su estado.
 2. Si un mensaje del bus se pierde (pub/sub no garantiza entrega), la resincronización periódica del snapshot corrige la pantalla en
-

menos de diez segundos.

3. Si el bus no llega a conectar, cada cliente cae a consultar el snapshot en bucle corto: se degrada la latencia, no la sincronía.
 4. Si el PIN no existe o la sesión terminó, la vista del público muestra la pantalla de cierre con el enlace de feedback, nunca un error crudo.
 5. Un texto de un segmento que no tenga forma de PIN devuelve el 404 normal del sitio sin llegar a consultar Redis.
-

Diagrama de secuencia

Los diagramas de secuencia describen el funcionamiento interno del sistema desde el punto de vista de la implementación y forman parte de la vista de interacción de UML (SENA, 2018). Cada objeto se dibuja como una caja en la parte superior de una línea vertical punteada, llamada línea de vida, que representa su existencia durante la interacción; cada mensaje es una flecha entre dos líneas de vida, y el orden de los mensajes transcurre de arriba hacia abajo.

Se distinguen dos tipos de mensajes. Los sincrónicos corresponden a llamadas a métodos en las que el emisor queda bloqueado hasta que la llamada termina, y se representan con flechas de cabeza llena. Los asincrónicos terminan de inmediato y crean un nuevo hilo de ejecución dentro de la secuencia; se representan con flechas de cabeza abierta, y la respuesta a un mensaje se dibuja con una flecha discontinua (SENA, 2018).

Las cuatro interacciones modeladas corresponden a los casos de uso de mayor riesgo del sistema: la autenticación del administrador, el ciclo de monitoreo que abre incidentes, la aplicación de las defensas de seguridad en cada petición y el procesamiento de un webhook de pago. Se eligieron esos cuatro porque son los flujos donde un error no degrada una función sino que compromete datos, dinero o disponibilidad.

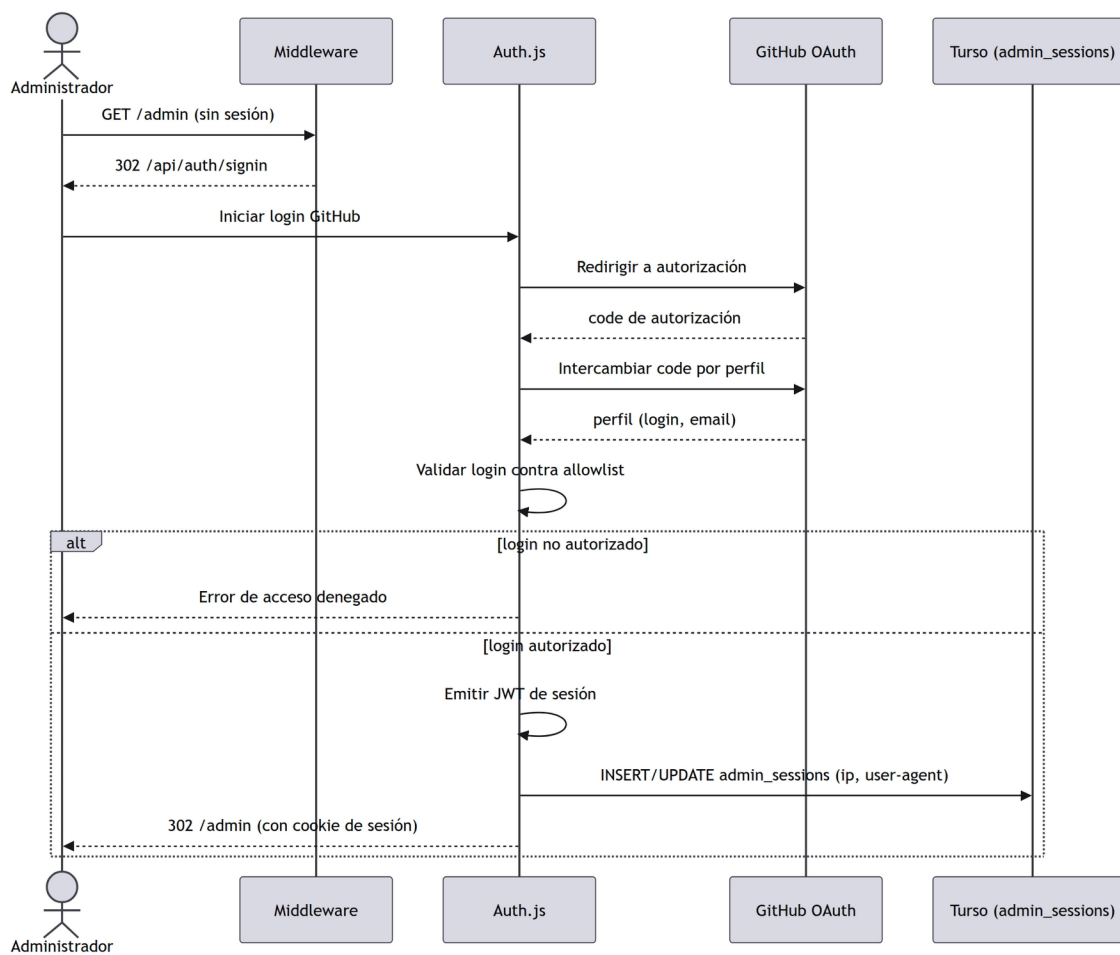
Figura 13*Login del administrador (GitHub OAuth)**Nota.* CU-04 · Autenticación con allowlist y registro de dispositivo.

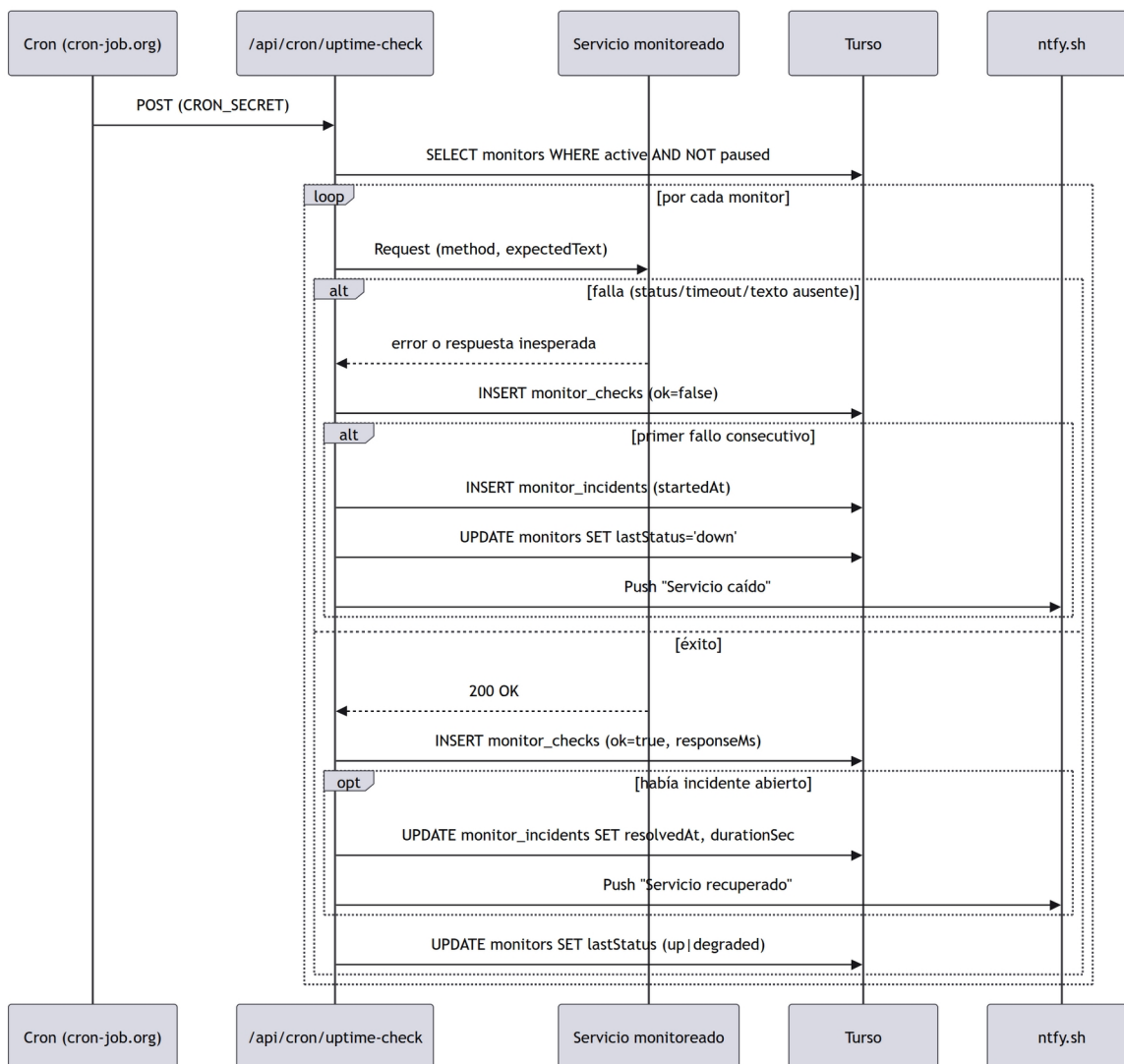
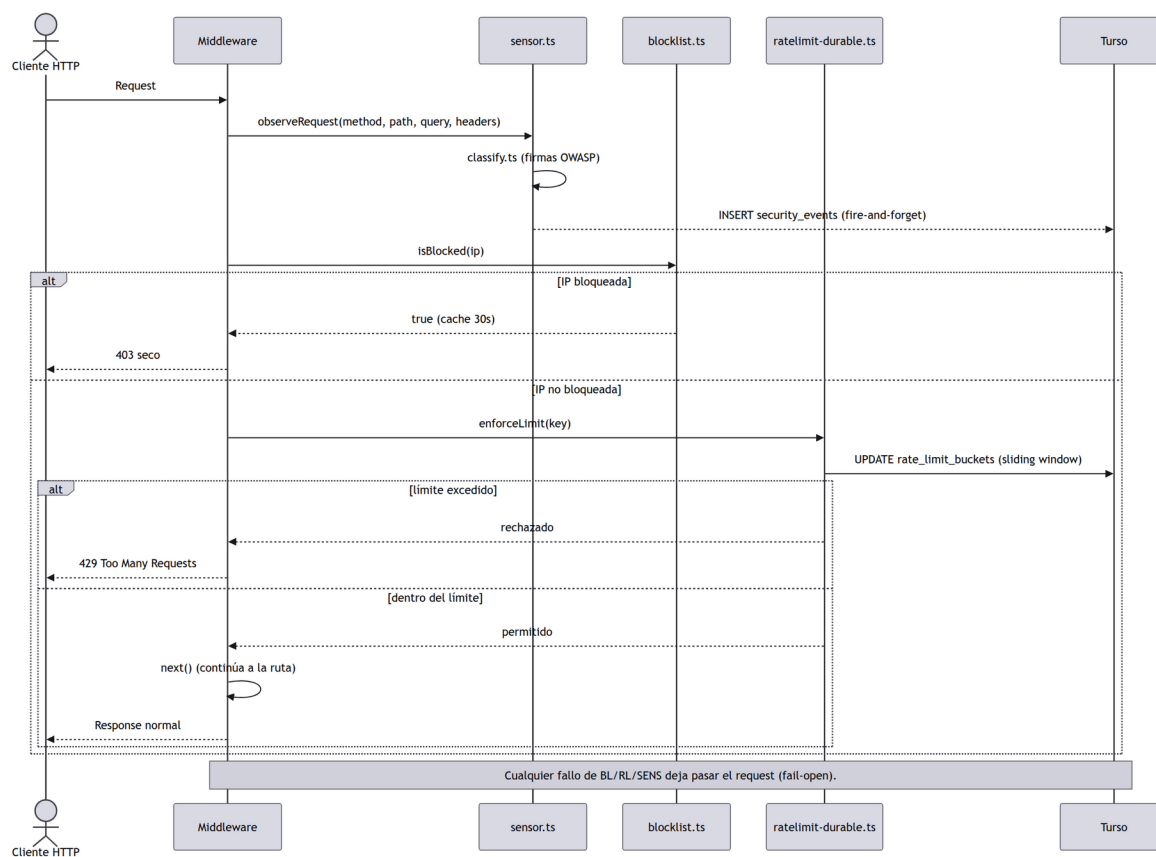
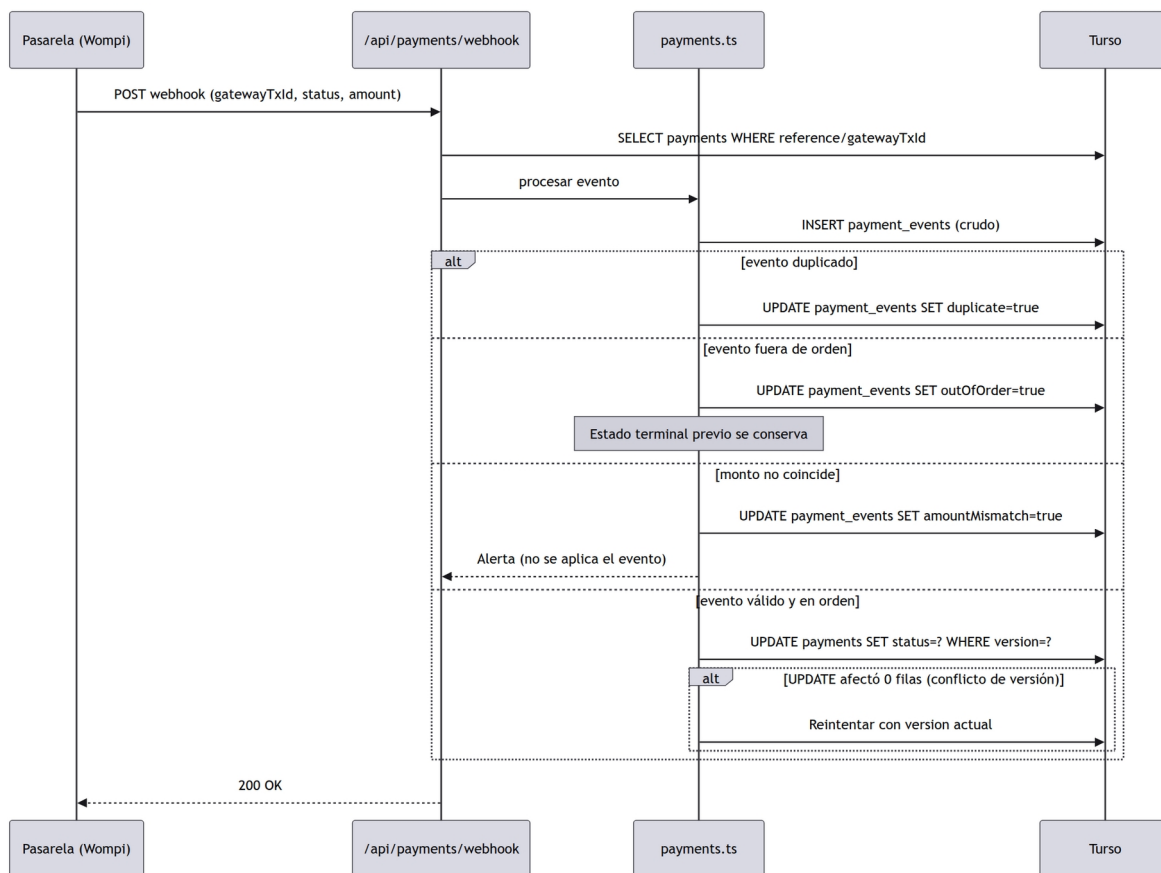
Figura 14*Chequeo de monitor y apertura de incidente**Nota.* CU-09 · Cron externo, sondeo HTTP y notificación push.

Figura 15*Enforcement de seguridad en el middleware*

Nota. CU-14 · Sensor, blocklist y rate limiting durable, todo fail-open.

Figura 16*Webhook de pago con idempotencia*

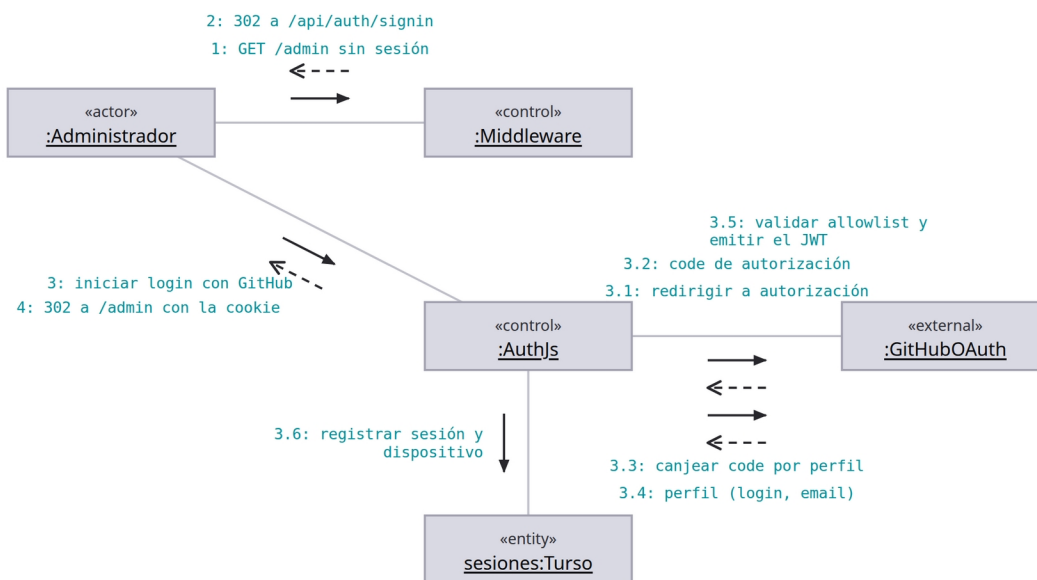
Nota. CU-12 · Estados sin retroceso y bitácora completa de eventos.

Diagrama de comunicación

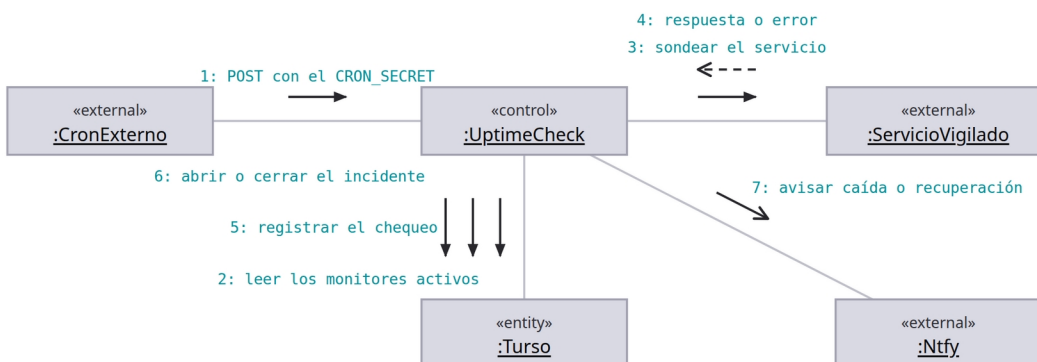
El diagrama de comunicación, llamado de colaboración en UML 1.0, transmite la misma información que un diagrama de secuencia. La diferencia está en el énfasis: el de secuencia destaca el orden en que se emiten los mensajes e incorpora la línea de vida de cada objeto, mientras que el de comunicación destaca la relación entre los objetos y genera una visión espacial del sistema durante la ejecución de un proceso (SENA, 2018).

Sus componentes gráficos son los del diagrama de secuencia salvo por dos diferencias: no se emplea la línea de vida y cada mensaje lleva un número que identifica el orden en que debe ejecutarse. La numeración es, por tanto, el único portador de la secuencia temporal.

Los cuatro diagramas siguientes representan las mismas interacciones de la sección anterior, vistas ahora por su estructura. El contraste entre ambas vistas es deliberado: la representación espacial revela con qué otros objetos se acopla cada participante, información que la vista temporal dispersa a lo largo del eje vertical.

Figura 17*Login del administrador (GitHub OAuth)*

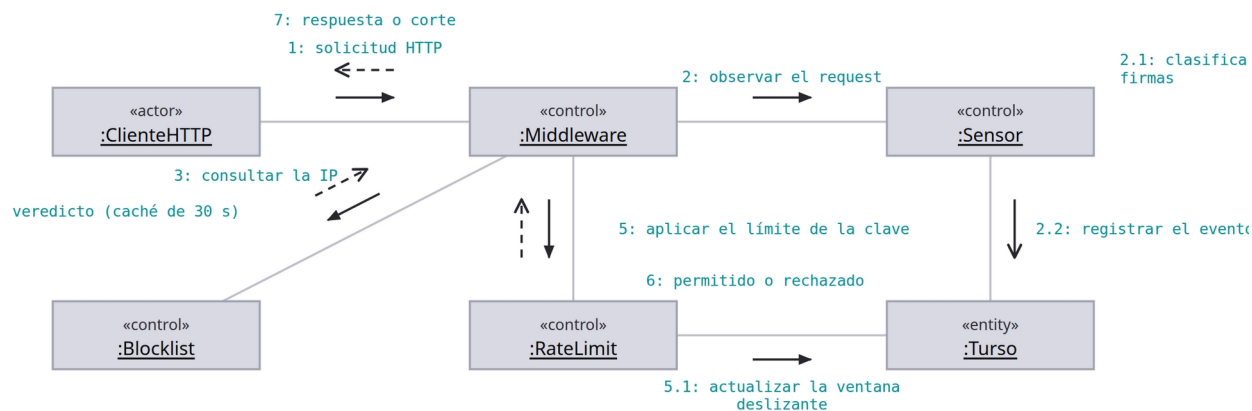
Nota. La misma interacción del diagrama de secuencia, vista por su estructura: qué objeto está enlazado con cuál. Se lee de un vistazo que el navegador nunca habla con GitHub ni con la base - solo con el middleware y con Auth.js.

Figura 18*Chequeo de monitor y apertura de incidente*

Nota. El ciclo de sondeo disparado por el cron externo. La estructura enseña algo que la secuencia no subraya: el endpoint es el único objeto enlazado con todos los demás, así que es también el único punto donde el ciclo puede romperse.

Figura 19

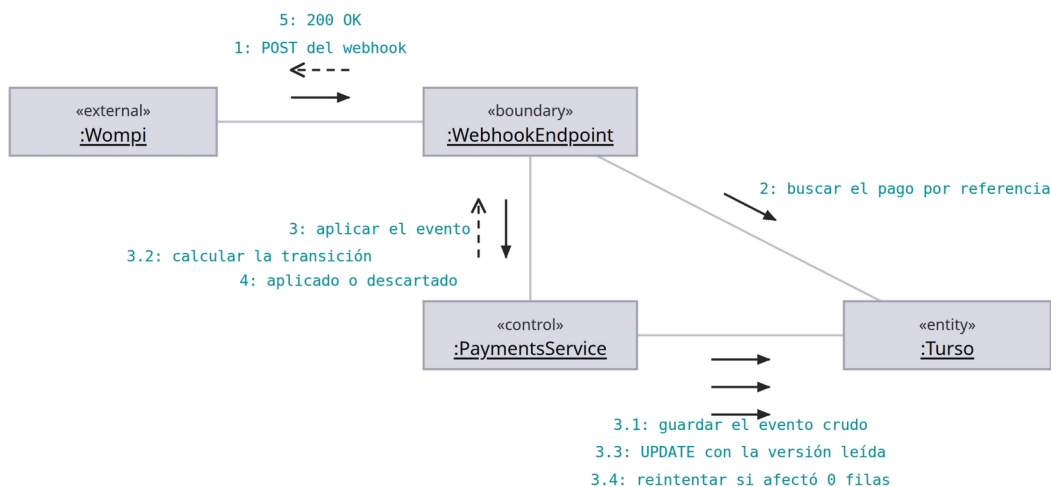
Enforcement de seguridad en el middleware



Nota. Los tres guardas de seguridad y sus enlaces. La vista estructural deja claro que el sensor no está en el camino del cliente: cuelga del middleware por un enlace propio y escribe en la base por su cuenta.

Figura 20

Webhook de pago con idempotencia



Nota. La misma interacción que el diagrama de actividades describe por dentro, aquí vista por sus enlaces. El endpoint no toca la máquina de estados ni ella responde a la pasarela: cada objeto habla solo con sus vecinos.

Diagrama de actividades

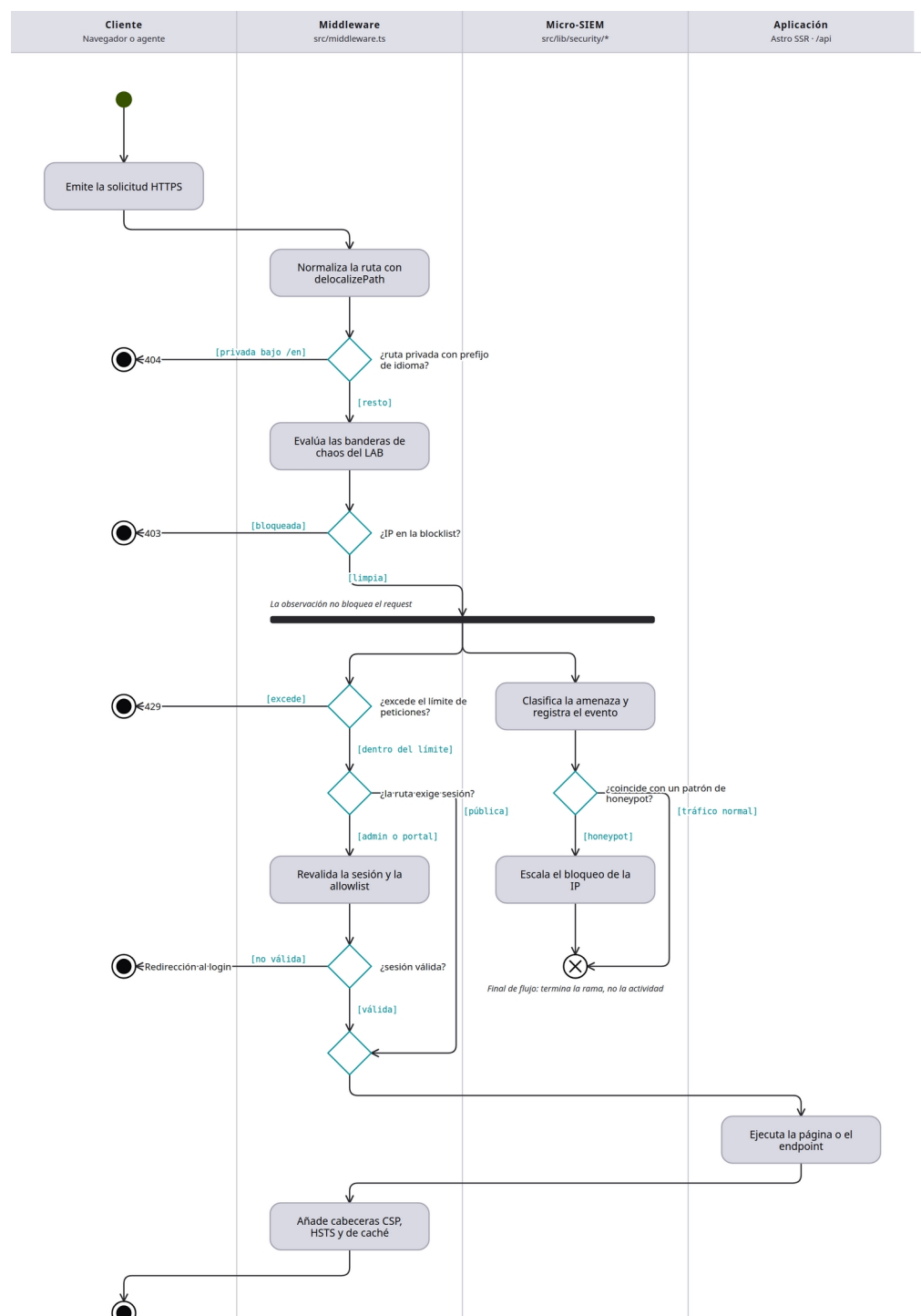
El diagrama de actividades muestra el flujo de actividades dentro de un sistema, cubre su parte dinámica y resalta el flujo de control entre objetos (SENA, 2018). Es similar al diagrama de flujo tradicional, con una diferencia decisiva: permite representar actividades concurrentes, es decir, pasos que se ejecutan en paralelo y luego se unen. En el contexto de la orientación a objetos se usa habitualmente para detallar los pasos lógicos que requiere un caso de uso.

La notación empleada es la siguiente. El inicio es un círculo relleno, y todo diagrama posee uno solo. El fin es un círculo que rodea a un círculo relleno. Las actividades son rectángulos de vértices redondeados, y las flechas que las enlazan se denominan transiciones. La decisión se representa mediante un rombo del que parten los caminos alternativos, cada uno con su condición. El inicio y el fin de una ruta concurrente se representan mediante una línea horizontal sólida (SENA, 2018).

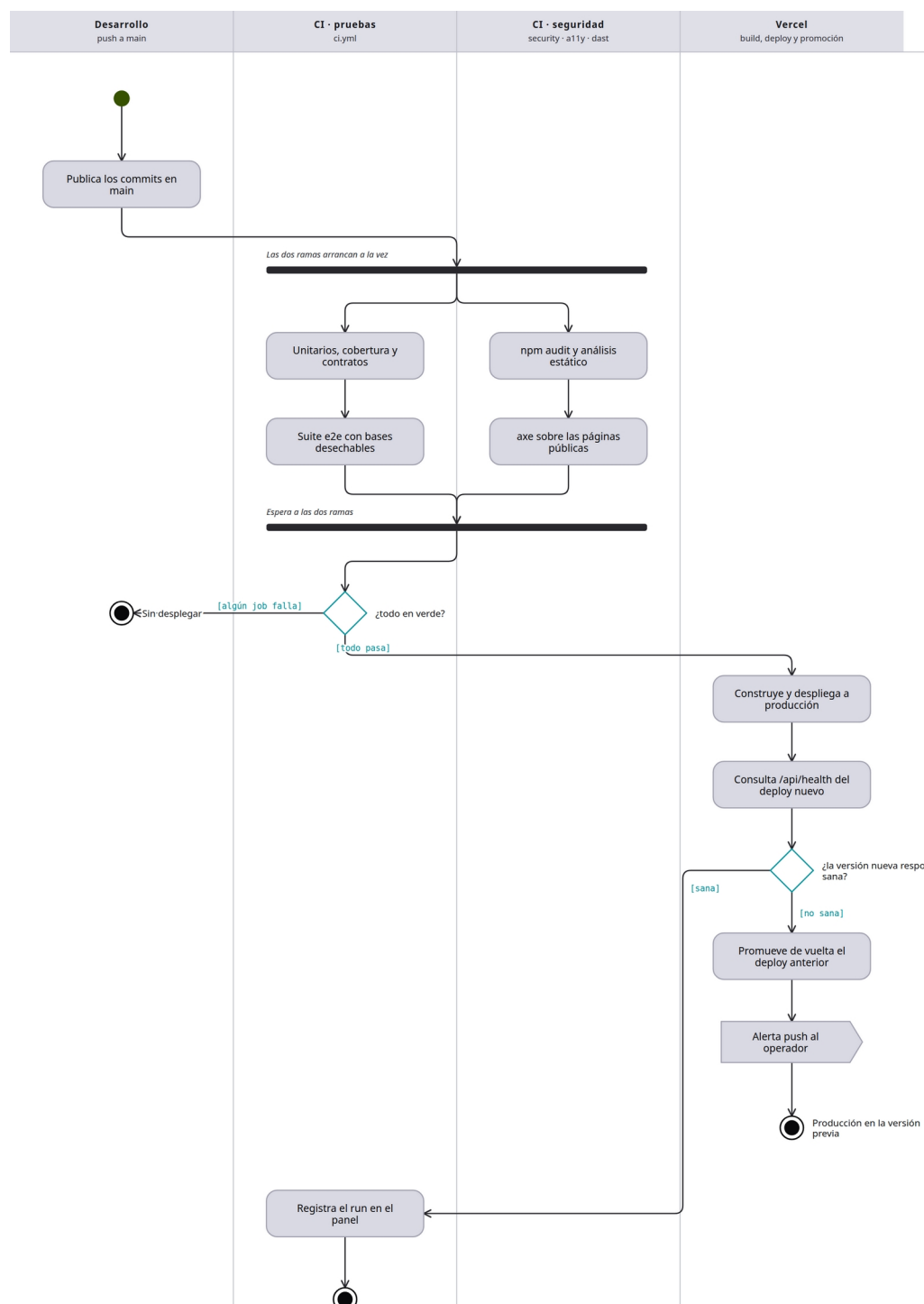
Se modelaron tres procesos: la atención de una solicitud en la capa intermedia de seguridad, el flujo de integración y despliegue continuo con reversión automática, y la aplicación idempotente de un evento de la pasarela de pagos. Los tres contienen bifurcaciones condicionales, y los dos primeros contienen además rutas concurrentes, lo que justifica el uso de este diagrama frente a un diagrama de flujo convencional.

Figura 21

Atención de una solicitud en el middleware



Nota. El camino completo de un request desde que entra al edge hasta que sale con sus cabeceras. Concentra en un solo punto la normalización de idioma, el chaos del LAB, la blocklist, el límite de peticiones y las dos autenticaciones, en ese orden.

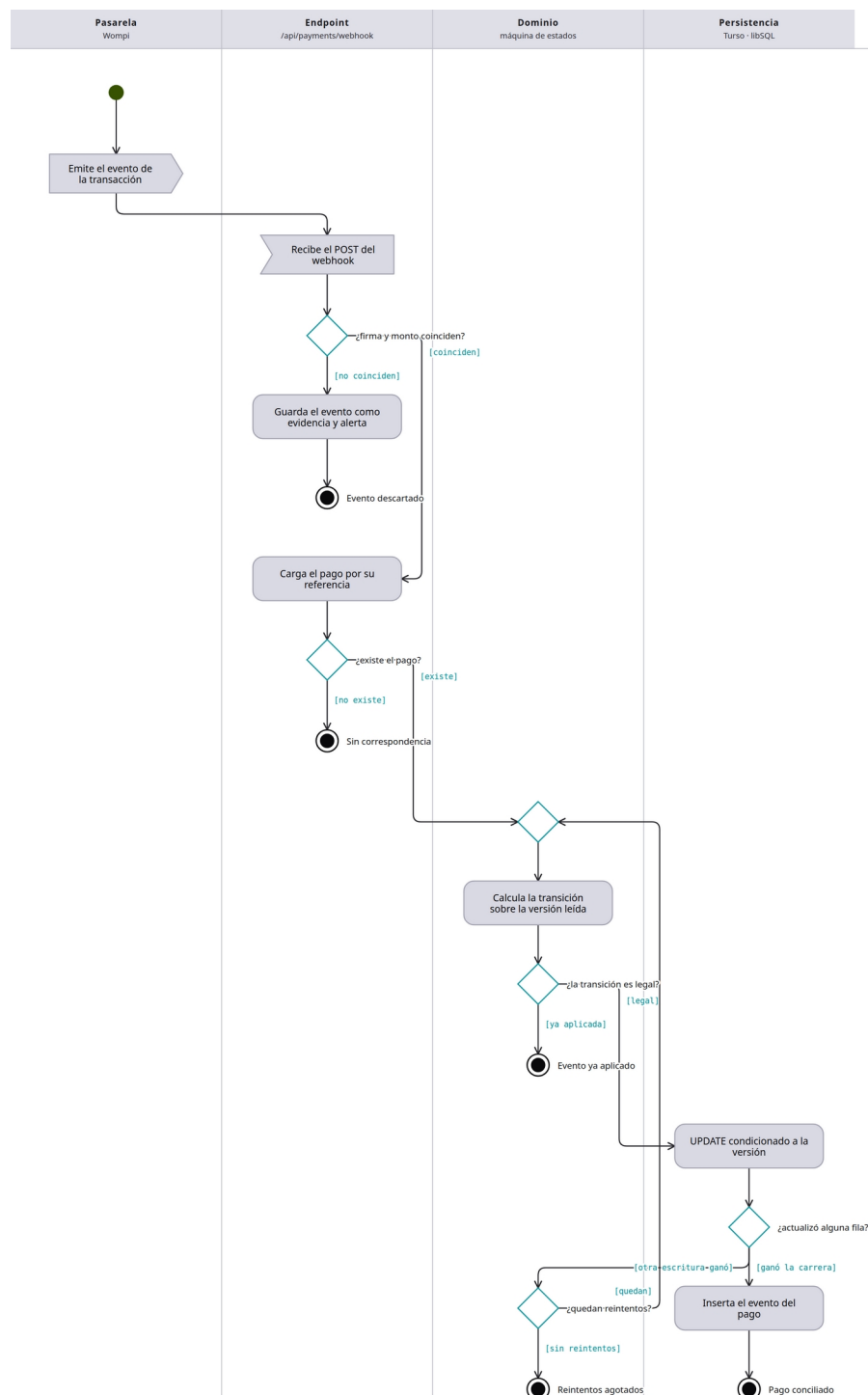
Figura 22*Integración, despliegue y verificación con rollback*

Nota. Del push a main hasta producción verificada. Las dos ramas de verificación (pruebas y seguridad/accesibilidad) corren concurrentes y se sincronizan antes de desplegar; después del deploy hay una etapa más, la que decide si la versión nueva se queda o se

revierte.

Figura 23

Aplicación idempotente de un evento de la pasarela



Nota. Qué ocurre dentro del sistema cuando llega un webhook de pago. No es el proceso de cobro (ese está en BPMN); es el algoritmo que garantiza que un evento repetido, reenviado o simultáneo no cobre dos veces ni deje el pago a medias.

Diagramas de estructura

Diagrama de clases

El diagrama de clases es el más común para describir el diseño de los sistemas orientados a objetos: muestra un conjunto de clases con sus atributos, operaciones, interfaces y relaciones (SENA, 2018). Una clase es la unidad básica que agrupa una colección de objetos con un mismo comportamiento, y se compone de tres partes: el nombre, los atributos o propiedades, y los métodos u operaciones, que se identifican con verbos en infinitivo.

La visibilidad de atributos y métodos se indica con un símbolo antepuesto. El signo más (+) corresponde a visibilidad pública, accesible desde cualquier punto; el signo menos (–) a visibilidad privada, accesible solo desde dentro de la clase; y la almohadilla (#) a visibilidad protegida, accesible desde la clase y sus subclases pero no desde fuera (SENA, 2018).

Las relaciones entre clases pueden ser de herencia, asociación, agregación, composición y dependencia. La composición expresa una relación estática de parte y todo, en la que el tiempo de vida del objeto incluido depende del que lo incluye; la agregación expresa una relación dinámica en la que ambos tiempos de vida son independientes; la asociación vincula objetos que colaboran sin que uno dependa del otro para existir; y la dependencia indica que una clase instancia o crea objetos de otra (SENA, 2018; Larman, 2007).

El modelo de datos del sistema se presenta dividido en cuatro vistas temáticas, no porque sean subsistemas aislados sino porque un único diagrama con todas las entidades resultaría ilegible en papel. Las vistas corresponden a la gestión comercial y financiera, la observabilidad y el laboratorio, la seguridad, y el currículo y la formación.

Figura 24

CRM y finanzas

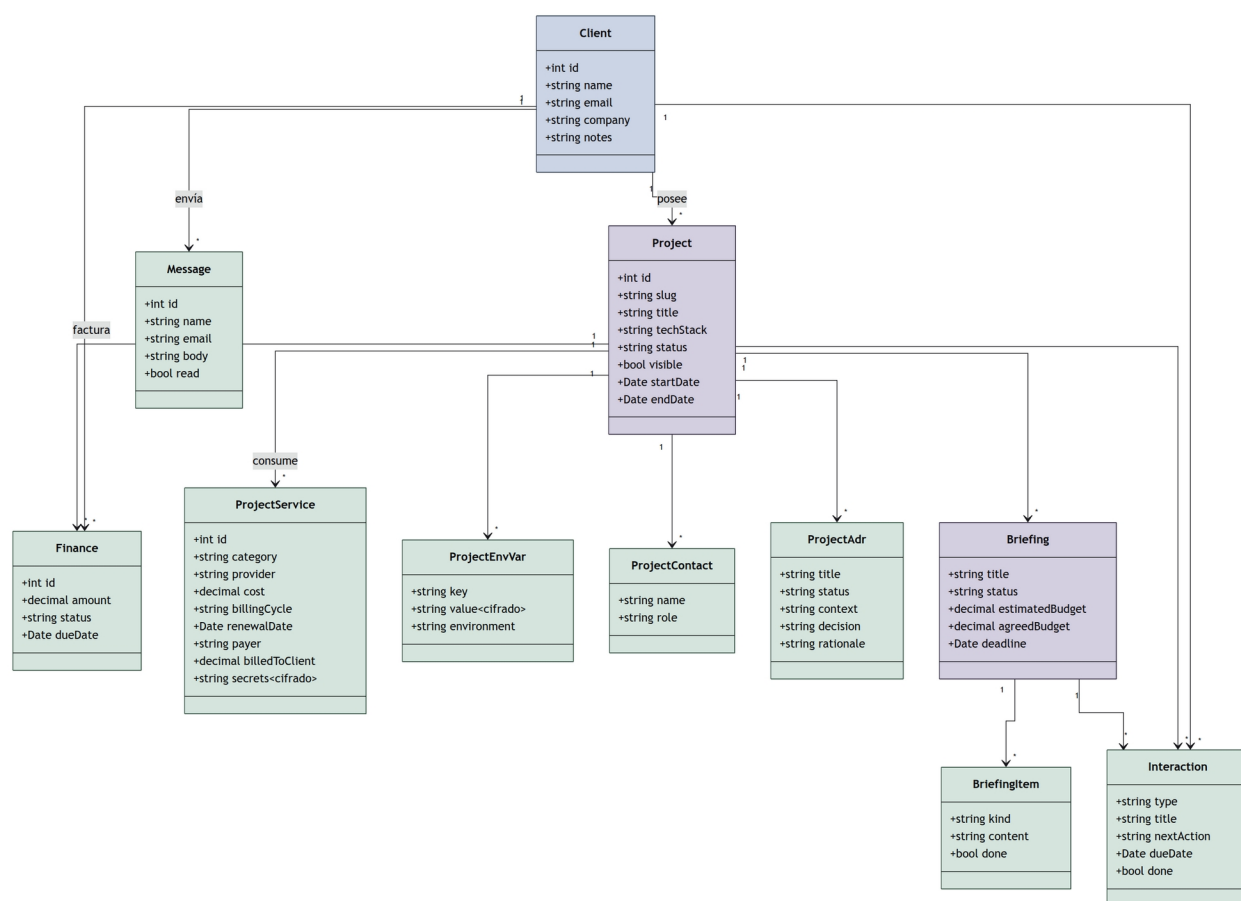


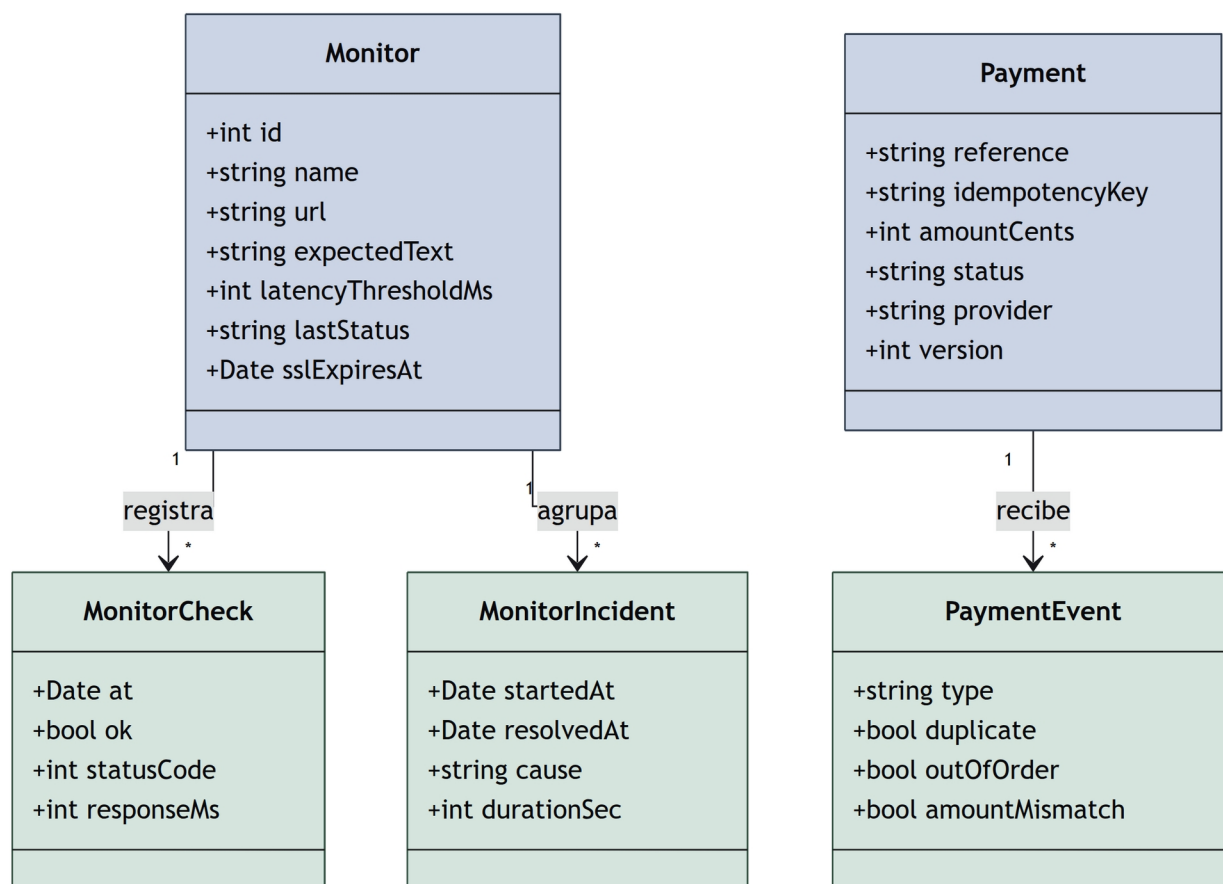
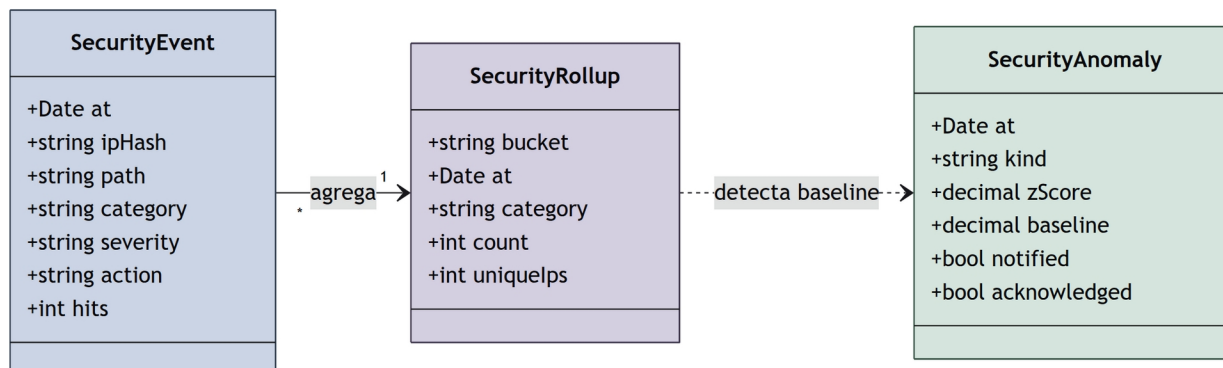
Figura 25*Observabilidad y pagos***Figura 26***Seguridad (micro-SIEM)*

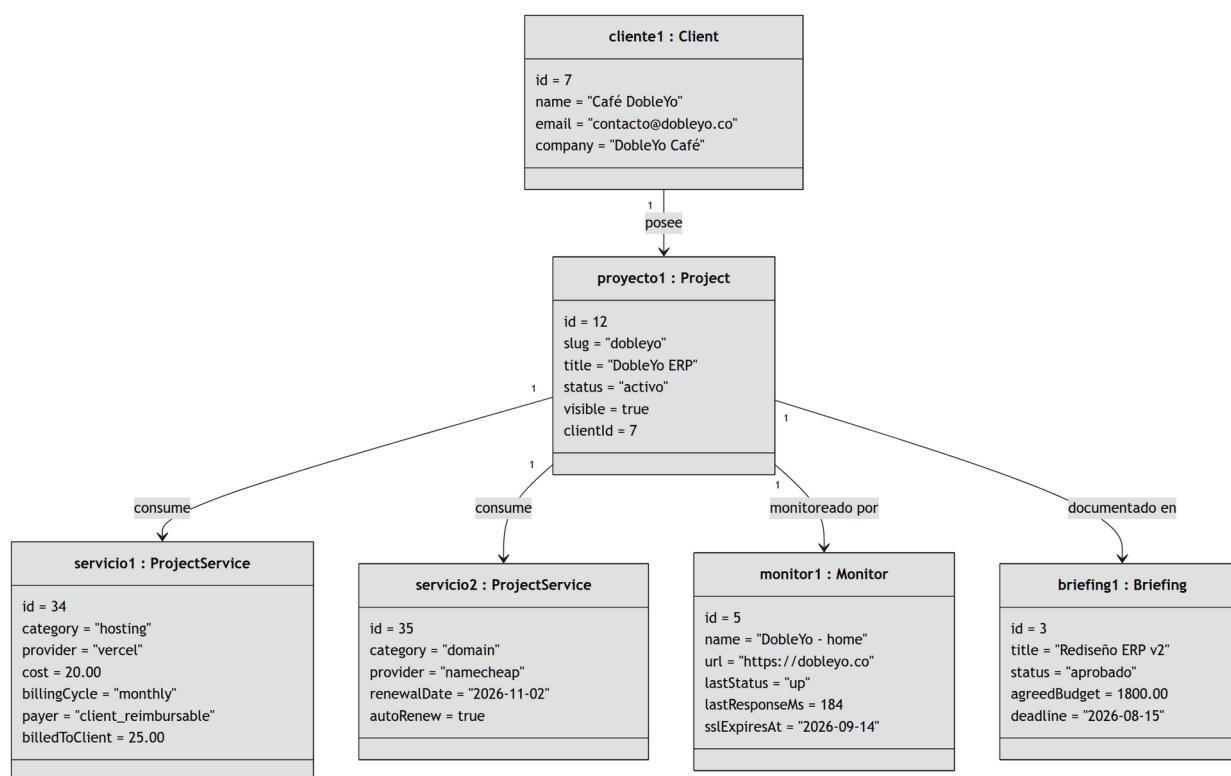
Diagrama de objetos

Un diagrama de objetos es en esencia muy similar a uno de clases, pero su propósito es modelar el sistema en un momento determinado a partir de las instancias derivadas de las clases (SENA, 2018). En lugar de presentar el nombre de la clase como una abstracción general, presenta un objeto concreto, lo que aporta claridad al modelo. Se emplean los mismos componentes que en el diagrama de clases, con la salvedad de que se omite la multiplicidad, pues esta se observa de manera explícita al incorporar varias instancias de una misma clase.

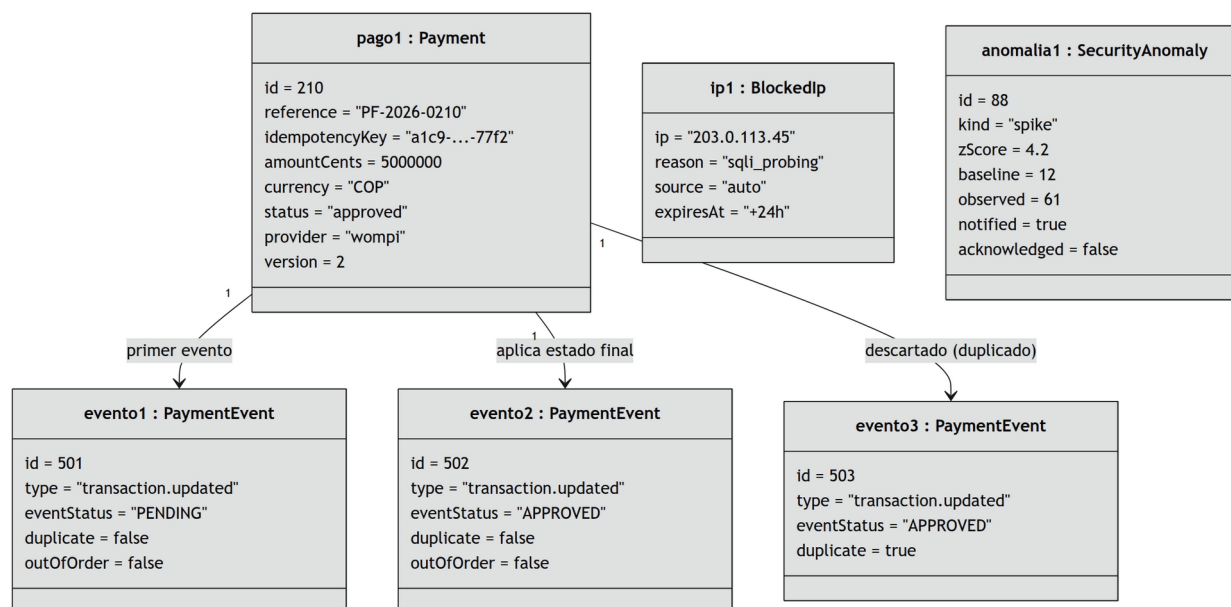
La sintaxis del nombre de un objeto es nombreObjeto:NombreClase. Las dos instantáneas que siguen se construyeron con datos que efectivamente ocurren en producción, y cumplen una función de verificación: si una configuración real no puede representarse con el diagrama de clases vigente, el modelo estructural está incompleto.

Figura 27

Instantánea 1 - Proyecto DobleYo con sus servicios



Nota. Un cliente con un proyecto activo, dos costos asociados, un monitor y un briefing aprobado.

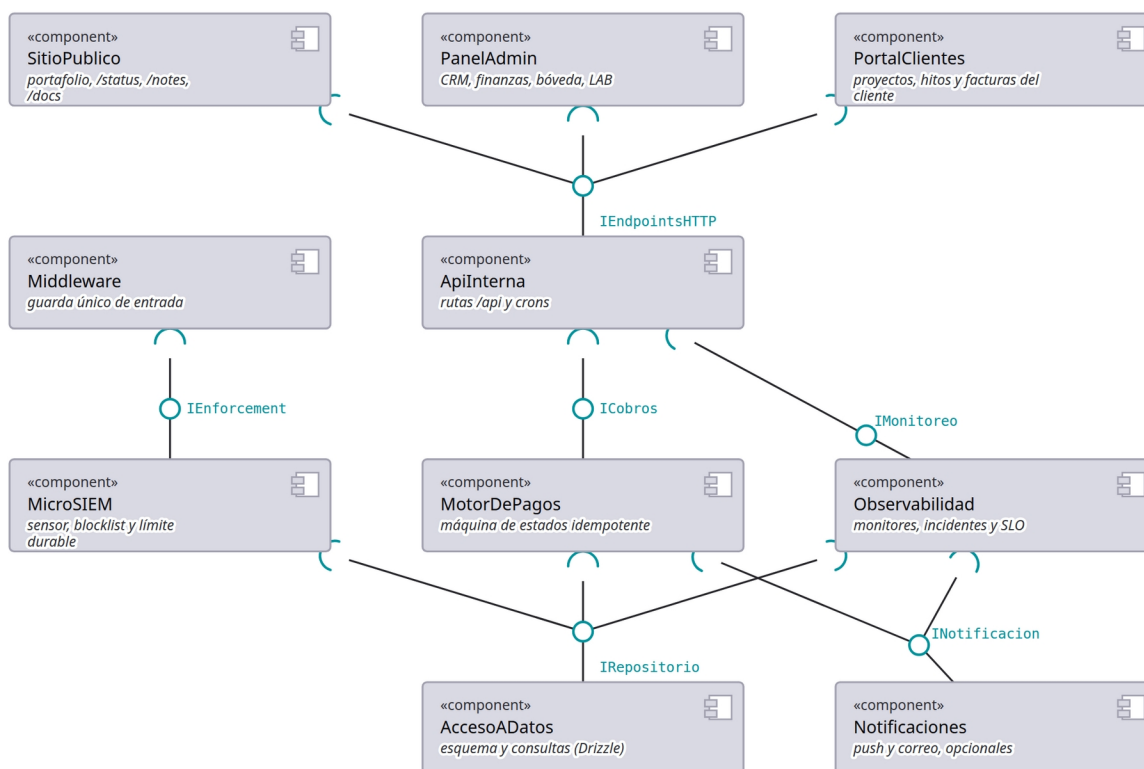
Figura 28*Instantánea 2 - Pago con evento duplicado + IP bloqueada*

Nota. Un pago aprobado cuya bitácora conserva el evento duplicado descartado (CU-12), junto a una IP bloqueada automáticamente y la anomalía que la originó (CU-14).

Diagrama de componentes

El diagrama de componentes representa las piezas que conforman una aplicación, junto con sus relaciones, interacciones e interfaces públicas (SENA, 2018). Modela el sistema a partir de las unidades desde las cuales se construye la aplicación, y resulta especialmente útil para comprender sistemas grandes como composición de partes pequeñas.

La notación se apoya en la metáfora de bola y enchufe: una interfaz proporcionada, aquello que el componente ofrece, se dibuja como un círculo sólido en el extremo de una línea; una interfaz requerida, aquello que el componente necesita, se dibuja como un semicírculo abierto hacia la interfaz con la que se acopla. Cuando la bola encaja dentro del enchufe se representa un conector de ensamblaje, es decir, una dependencia real entre los dos componentes.

Figura 29*Componentes y sus interfaces*

Nota. Las piezas de software del sistema y las interfaces por las que dependen unas de otras. Lo que el diagrama afirma es sustituibilidad: un componente que solo depende de interfaces provistas puede cambiarse por otro que ofrezca las mismas sin tocar a sus consumidores.

Diagrama de despliegue

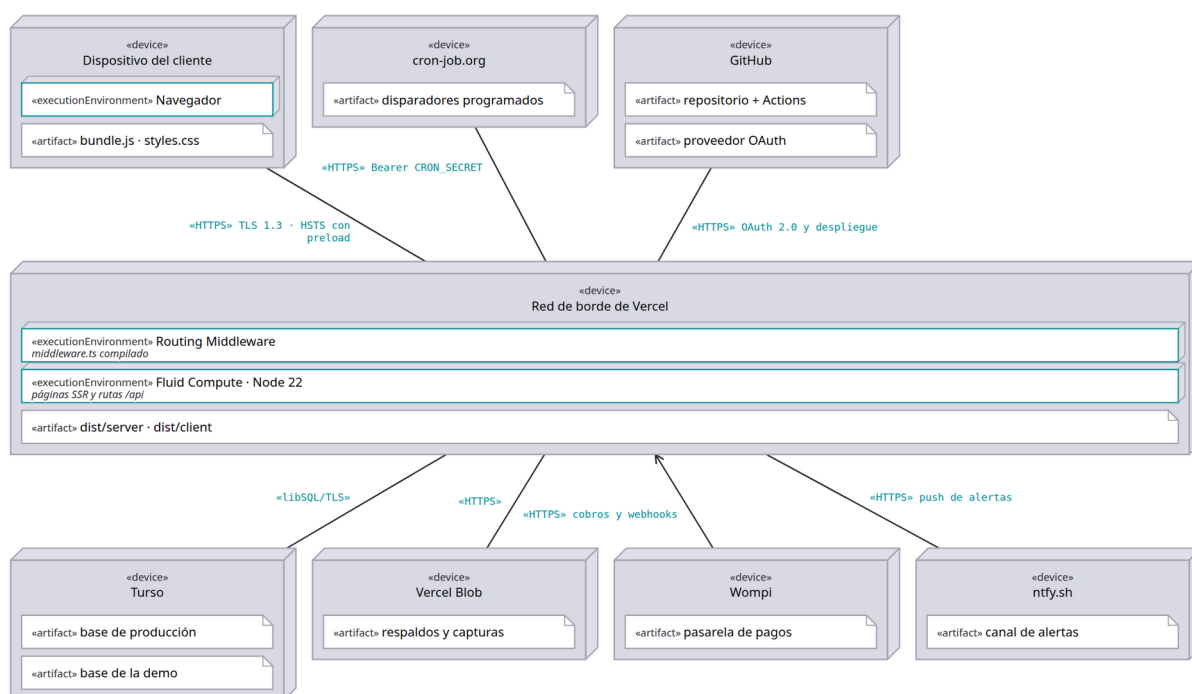
El diagrama de despliegue físico muestra cómo y dónde se desplegará el sistema. Las máquinas físicas y los procesadores se representan como nodos, visualizados habitualmente como cubos, y los artefactos se ubican dentro de esos nodos para modelar el despliegue (SENA, 2018). La ubicación de cada artefacto se guía por las especificaciones de despliegue.

El caso de este sistema tiene una particularidad que el diagrama expone con claridad: no existe un servidor propio. El cómputo ocurre en funciones administradas por

la plataforma de despliegue, la base de datos reside en un servicio gestionado con réplicas por región, y las tareas programadas las dispara un planificador externo mediante peticiones autenticadas. Los nodos del diagrama son, por tanto, entornos de ejecución de terceros y no máquinas bajo control directo del proyecto.

Figura 30

Despliegue de producción (codebymike.tech)

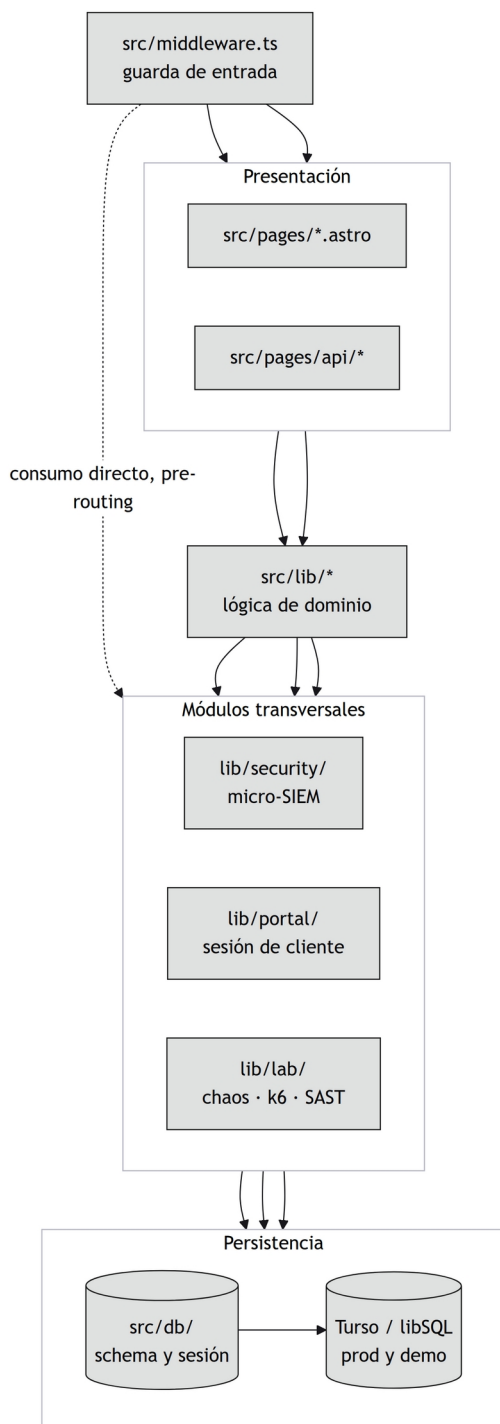


Nota. Qué corre dónde y por dónde viaja cada cosa. El sistema no tiene servidor propio: todo el cómputo vive en la red de borde de Vercel, la persistencia es un servicio gestionado y los disparadores periódicos vienen de fuera, no de un proceso residente.

Diagrama de paquetes

El diagrama de paquetes permite organizar los elementos de modelado en agrupaciones y representar las dependencias entre ellas (SENA, 2018). Sus usos más frecuentes son ordenar diagramas de casos de uso y de clases, aunque no está limitado a esos elementos.

En este sistema el diagrama de paquetes documenta una regla de arquitectura que el código debe respetar: las dependencias fluyen en una sola dirección, desde las páginas y los puntos de entrada hacia la lógica de dominio y de ahí hacia el acceso a datos, sin ciclos de retorno. Los módulos de lógica pura no dependen de la base de datos, y esa restricción es la que permite probarlos sin levantar una base de datos real.

Figura 31*Diagrama de paquetes*

Conclusiones

El modelado con UML del sistema CodeByMike permitió representar, en una notación estándar, tanto las funcionalidades que la solución ofrece a sus actores como la estructura interna que las soporta. Los diagramas de comportamiento evidenciaron que los flujos de mayor riesgo del sistema (autenticación, monitoreo, defensa perimetral y procesamiento de pagos) comparten un mismo principio de diseño: ninguna falla en un mecanismo auxiliar debe interrumpir el flujo principal.

El contraste entre el diagrama de secuencia y el de comunicación resultó particularmente útil. Aunque ambos representan la misma interacción, la vista espacial dejó ver acoplamientos entre objetos que la vista temporal distribuye a lo largo del eje vertical y, por tanto, oculta. Esto confirma la observación de la guía en el sentido de que la elección del diagrama no es una cuestión de preferencia sino de qué pregunta se quiere responder (SENA, 2018).

Por último, generar los diagramas desde el código fuente en lugar de dibujarlos en una herramienta aparte cambió su función dentro del proyecto: dejaron de ser documentación que envejece para convertirse en una vista del sistema que se actualiza con él. Un diagrama que se desincroniza del código no solo pierde utilidad, sino que induce a error a quien lo consulta.

Referencias

- Fowler, M., & Scott, K. (1997). *UML gota a gota*. Pearson Educación.
- Gutiérrez, C. (2011). *Casos prácticos de UML*. Editorial Complutense.
- Kimmel, P. (2008). *Manual de UML: guía de aprendizaje*. McGraw-Hill Professional Publishing.
- Larman, C. (2007). *UML y patrones: una introducción al análisis y diseño orientado a objetos y al proceso unificado* (2.^a ed.). Prentice Hall.
- Object Management Group. (2017). *OMG unified modeling language (OMG UML), version 2.5.1*. <https://www.omg.org/spec/UML/2.5.1/>
- Rumbaugh, J., Jacobson, I., & Booch, G. (2007). *El lenguaje unificado de modelado: manual de referencia* (2.^a ed.). Addison-Wesley.
- Servicio Nacional de Aprendizaje. (2018). *Introducción al lenguaje de modelado UML*. SENA, Centro Industrial de Mantenimiento Integral.